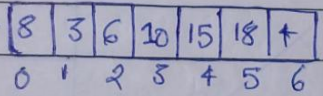
Hashing Technique

1. Why Hashing
2. Ideal Hashing
3. Modulus Hash Function
4. Drawbacks
5. Solutions

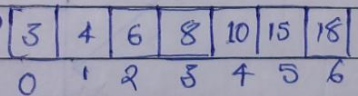
1. Why Hashing

Keys:- 8, 3, 6, 10, 15, 18, 4

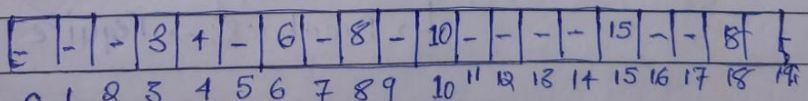
1. Linear — O(n)



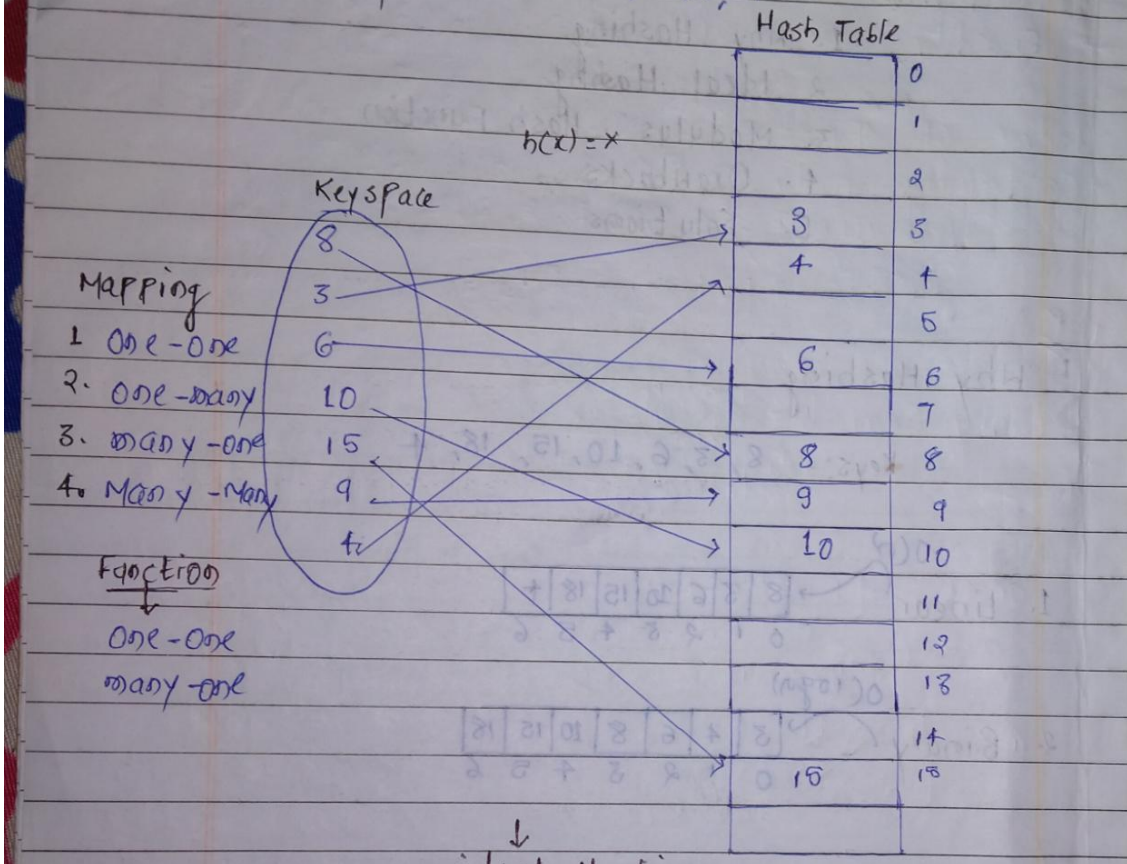
2. Binary — O(log n)



O(1)



Keys:- 8, 3, 6, 10, 15, 9, 4



Mapping
1. One-One
2. One-many
3. many-one
4. Many-Many

Functions
- One-One
- many-one

↓

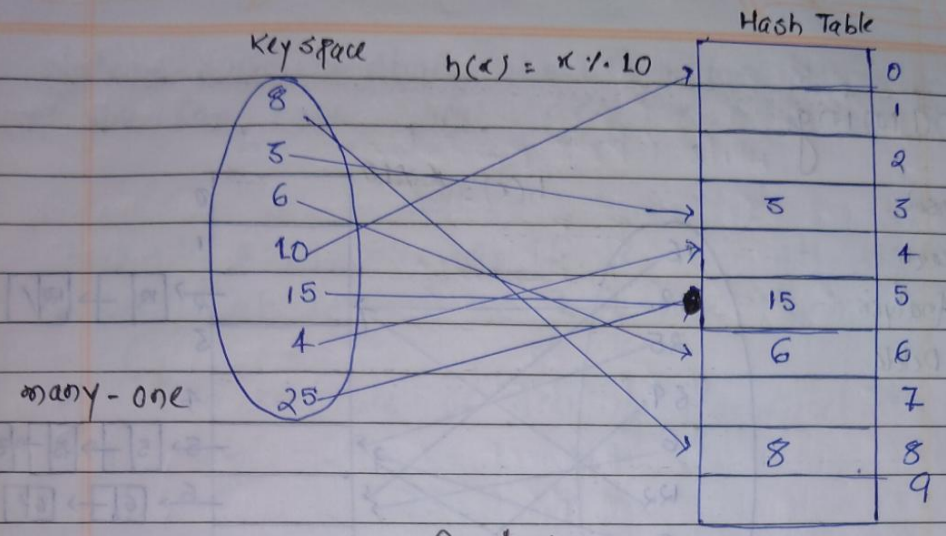
ideal Hashing

↳ Drawbacks → require lot of space

Modifying ideal Hashing

↓

Modulus Hash Function



Drawback

↓

collision

Solution [of collision problem]

Open Hashing ~ require extra space

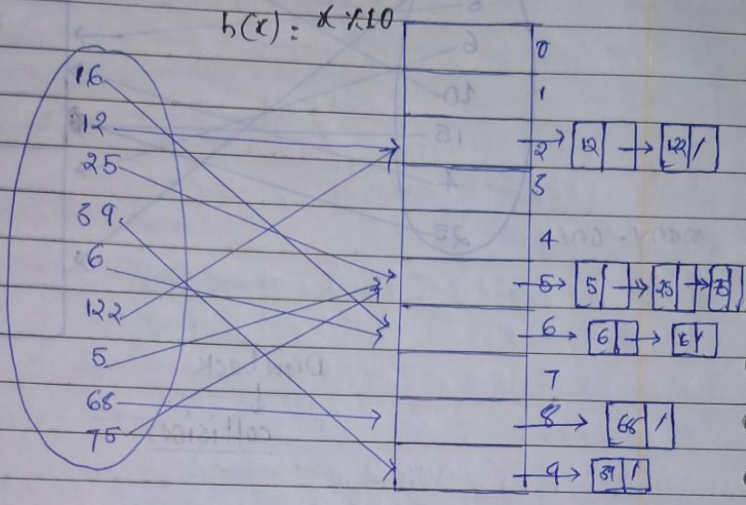
- chaining

Closed Hashing ~ doesn't require extra space

- Open Addressing
  1. Linear probing
  2. Qudratic Probing
  3. Double Hashing

Chaining

1. Insert
2. Search
3. Analysis
4. Delete



Search
Key = 75 ✓
Key = 12 ✓
Key = 65 ✗
Key = 10 ✗

→ First go at that index & then search in Node

Load Factor → $\lambda = n/size$

$n = 100$, $size = 10$ $\quad \lambda = \frac{100}{10} = 10$

Avg. Successful search

$t = 1 + \lambda/2$ → avg time for searching in Node

(1 ✓ for going for that index)

Avg. Unsuccessful search

$t = 1 + \lambda$

Note:- you can change hashFunction, it is upon you you can take $h(x) = (x/10) * 10$

Profit

Keys → 5, 35, 95, 65, 75, 45, 125 → all will be stored at index 5 if you take $h(x) = x/10$, ~~then~~

So keep care of deciding HashFunction based on data

→

```
#IFndef chains_h
# define chains_h

struct Node
{   int data;
    struct Node *next;
};

Void SortedInsert (struct Node **H, int x)
{
    struct Node *t, *q = NULL, *p = *H;

    t = (struct Node*) malloc (sizeof (struct Node));
    t->data = x;
    t->next = NULL;

    IF (*H == NULL)
        *H = t;
```

```
else
{
    while (P && P->data<x)
    {
        q=P
        P=P->next;
    }
    if (P==First)
    {
        t->next = First;
        First = t;
    }
    else
    {
        t->next = q->next;
        q->next = t;
    }
}

struct Node *search (struct Node *P, int key)
{
    while (P != NULL)
    {
        if (key == P->data)
        {
            return P;
        }
        P = P->next;
    }
    return NULL;
}

# endif /* chains_h */
```

```
#include <stdio.h>
#include "chains.h"

int main()
{
    struct Node *HT[10];
    int i;

    For (i=0; i<10; i++)
        HT[i] = NULL;
    Insert(HT,12);
    Insert(HT,22);
    Insert(HT,42);
    temp = Search(HT[hash(21)],21); PrintF("%d", temp->data);
}                                                  ↓
                                                   22
int hash(int key)
{
    return key % 10;
}

void insert(struct Node *H[], int key)
{
    int index = hash(key);
    SortedInsert(&H[index], key);
}
```

Linear Probing

$h'(x) = (h(x) + F(i)) \% 10$, where $F(i) = i$, $i = 0, 1, 2, \ldots$

Insertion

$h(x) = x \% 10$

Keyspace: 26, 30, 45, 23, 25, 43, 74, 19, 29

0	30
1	29
2	
3	23
4	43
5	45
6	26
7	25
8	74
9	19

$h'(25) = (h(25) + F(0)) \% 10$
$= (5 + 0) \% 10 = 5$

$h'(25) = (h(25) + f(1)) \% 10$
$= (5 + 1) \% 10 = 6$

$h'(25) = (h(25) + f(2)) \% 10$
$= (5 + 2) \% 10 = 7$

$h'(29) = (h(29) + f(0)) \% 10$
$= (9 + 0) \% 10 = 9$
$= (9 + 1) \% 10 = 0$
$= (9 + 2) \% 10 = 1$

Search

Key = 45 ✓
Key = 74 ✓
Key = 40 X → [search untill key is Found or you get a Free space]

Search by seq ←

Analysis

Loading Factor $\lambda = \frac{n}{size}$

$\lambda = \frac{9}{10} = 0.9$

***

$\lambda \leq 0.5$ means half Filling

* Above example has been taken for Illustration Purpose, $\lambda = 0.9$ is Wrong

Avg. successful search

$$t = \frac{1}{\lambda} \ln\left(\frac{1}{1-\lambda}\right)$$

Avg. Unsuccessful Search

$$t = \frac{1}{1-\lambda}$$

Drawback:-

(i) As λ should always be $\boxed{\lambda \leq 0.5}$

↓

So space wasted

(ii) Linear probing has problem of cluster of key

For searching 74

We are going through

75, 26, 25

↑ cluster

Deletion:-

0	30
1	29
2	2
3	23
4	43
5	~~46~~
6	26
7	25
8	74
9	19

Delete 45

While deleting you have to do lot of change [have to take care of index]

↓

It is better to remove all data & Fill once again, it is called rehashing

↓

Time Consuming

* Deletion is not suggested for linear probing.

→
```
#include<stdio.h>
#define SIZE 10

int hash(int key)
{
    return key%SIZE;
}

int Probe(int H[], int key)
{
    int index = hash(key);
    int i=0;                          i*i → for quadratic probing at all places
    while(H[(index+i)%SIZE]!=0)
        i++;
    return (index+i)%SIZE;
}

void insert(int H[], int key)
{
    int index = hash(key);

    IF(H[index]!=0)
        index = Probe(H, key);
    H[index] = key;
}
```

```
int search (int H[], int Key)
{
    int index = hash (Key);
    int i=0;

    While (H[(index+i) % size] != Key)
        i++;

    return (index+i) % size;
}

int main()
{
    int HT[10] = {0};

    insert ( HT, 2);
    insert (HT, 25);
    insert (HT, 35);
    insert (HT, 26);

    Printf ("In Key Found at %d \n", search (HT, 35));

    return 0;
}
```

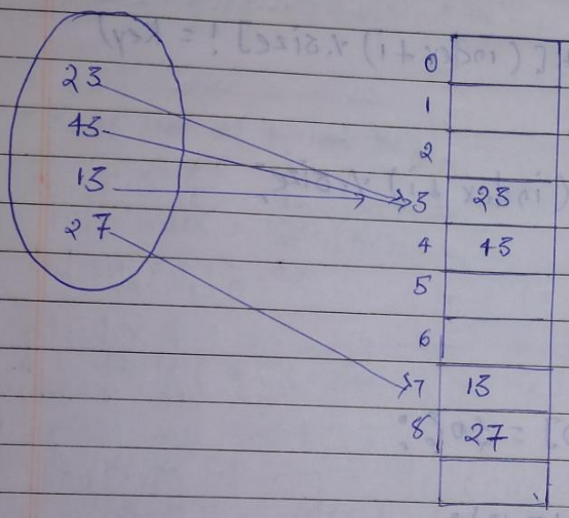
Quadratic probing

Was introduced to remove clustering

$h'(x) = (h(x) + f(i)) \% 10$

where $F(i) = i^2$ $i = 0, 1, 2 \ldots$

Key space $h(x) = x \% 10$ Hash Table



$h'(13) = (h(13) + f(0)) \% 10$

$(3+0) \% 10 = 3$

$(3+1) \% 10 = 4$

$(3+4) \% 10 = 7$

7 + 0

7 + 1

7 + 4

+9

Avg. successful search

$$\frac{-\log_e(1-\lambda)}{\lambda}$$

Avg successful search

$$\frac{1}{1-\lambda}$$

Double Hashing

$h1(x) = x \% 10$

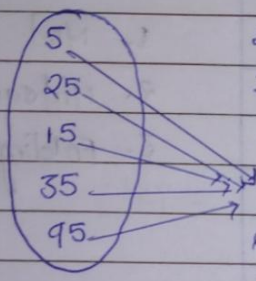
$h2(x) = 7 - (x \% 7)$ (nearest Prime no. of size)

$h'(i) = (h1(x) + i * h2(x)) \% 10$

where $i = 0, 1, 2 \ldots$

$h1(x) = x \% 10$ | H.T

Key space



0	
1	15
2	85
3	
4	95
5	5
6	
7	
8	25
9	

$h'(25) = (5 + 1*3) \% 10 = 8$

$7 - (25 \% 7)$

$7 - 4 = 3$

$h'(15) = (5 + 1*6) \% 10 = 1$

$7 - (15 \% 7)$

$7 - 1 = 6$

$h'(85) = (5 + 1*7) \% 10 = 2$

$7 - (35 \% 7)$

$7 - 0 = 7$

$h'(95) =$

$(5 + 3*5) \% 10 = 4$

$7 - (95 \% 7)$

$7 - 4 = 3$

Hash Function ideas

size = 9

H.T

0	
1	
2	
3	
4	
5	
6	
7	
8	

1. Mod
2. Midsquare
3. Folding

① $h(x) = (x \% size) + 1$

↑ Take size as Prime no. as it will reduce no. of collision

② Whatever is digit, do square of that & take middle digit

Ex:- Key = 11

$= (11)^2 = 121$

↑

Key = 13

$= (13)^2 = 169$

↑

Square of anyno is 6 digit = - - - - - -

↑

↳ You can take two digit (If two digit is not available then you can perform mod on that)

③ Folding

key = 12 33 47 → six digit key

12
33
47

92

key = "ABC"

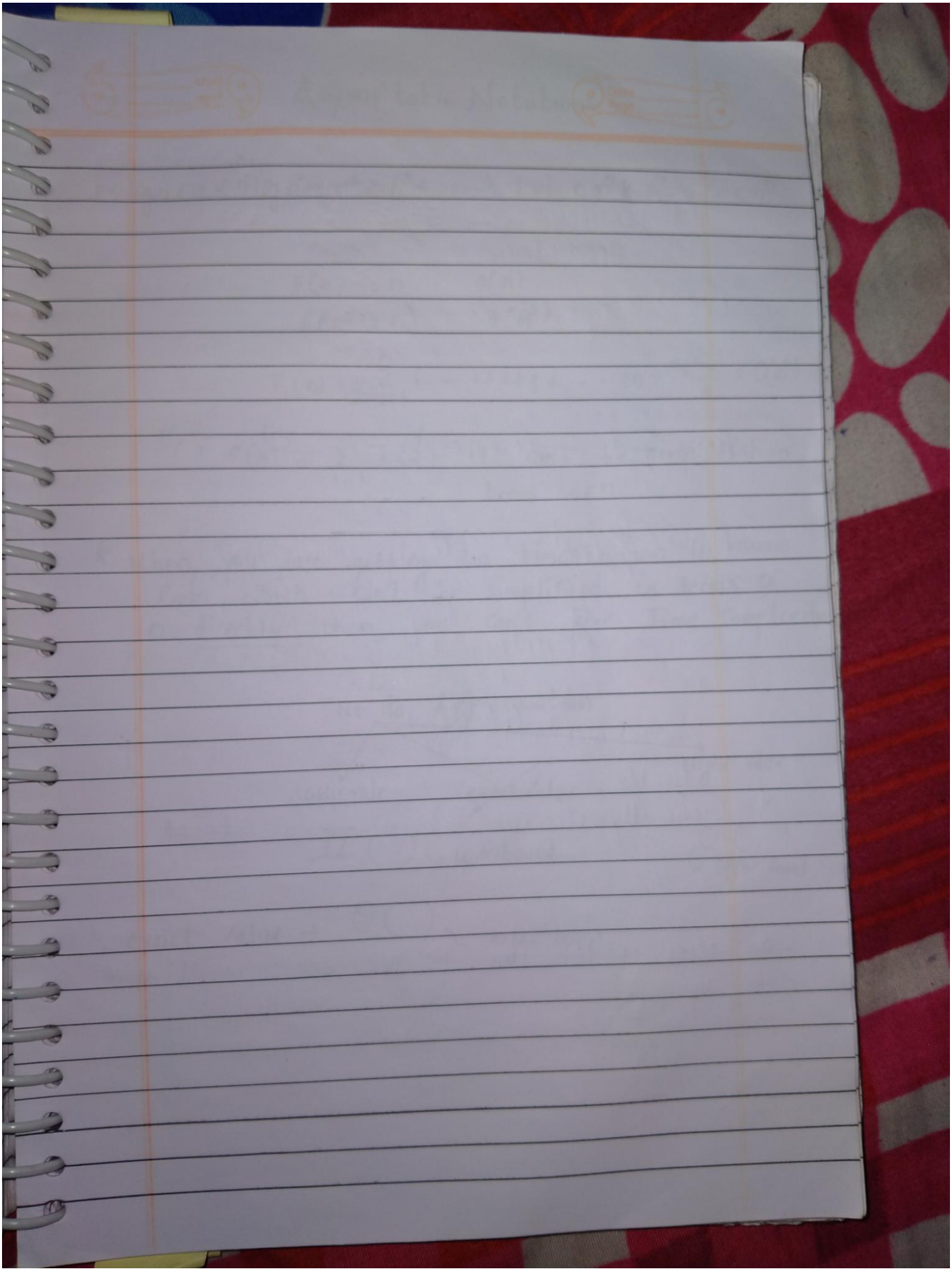
A B C

[65 66 67]

65
66
+67

198.

★ you can design your has Function



STL

WWW.bmlw.net

Template → used For genoring program
↓
For multiple data types

Solution → use generic Programming

```
Class List
{
   int a[10];
   —
   —
};
List l1
```

IF we want to store float, double then we have to make other class

```
template <class T>
class List
{
   T a[10];
   —
   —
};
List<float> l1
```

→ STL have Following three- well structured Compone

- Containers
- Algorithms
- iterators

Containers

Ex→ Vector can be used For creating dynamic arrays of char, integer, float and other types

Containers are classes used to store various data types.

Algorithms ~ Class that performs algorithm

Algorithms act on Containers.

iterators

—

Containers

Container library is a collection of classes

Common Containers

- vector : replicate array
- queue : queue
- stack : stack
- Priority_queue : heaps
- list : linked list
- set : Trees
- map : associative arrays

⋮

Array in STL
↓ having fixed size

→ # include < array >

→ array < int, 4 > obj ;
↑ size of array.

Member function

→ at
→ [] operator
→ Front() → return First element
→ back() → returns back element
→ Fill()
→ swap()
→ size()
→ begin()
→ end()

Ex:- array < int, 8 > data_array = { 11, 22, 33, 44, 55 };

at
↓
cout << data_array.at(2); ~ if greater than index, exception is thrown

[]
↓
cout << data_array[3];

Front()
↓
data_array.front(); → 11

back()
↓
data_array.back(); → 55 0 as size is 5

Fill()

```
array<int,8> data_array = {11,22,33,44,55};
dat_array.Fill(10);
```

// it will Fill the whole array with 10)

10	10	10	10	10	10	10	10
0	1	2	3	4	5	6	7

Swap()

```
array<int,5> data_array1 = {11,22,33,44,55};
array<int,5> data_array2 = {1,2,3,4,5};
data_array1.swap(data_array2);
For(int i=0; i<=7; i++)
    cout<< data_array1[i];
For(int i=0; i<=7; i++)
    cout<< data_array2[i];
```

→ 1 2 3 4 5
11 22 33 44 55

begin() → returns the iterator pointing to the first element of the array

end() → returns an iterator pointing to an element next to the last element in the array

Pair in STL

it is class

Pair <T1, T2> Pair1;

ex:- Pair <string, int> P1;

→ #include <iostream>, #include <Pair>
using namespace std;

```
class student{
 Private:
      string name;
      int age;
  Public:
      void Setstudent (string a, int a)
     { name = a; age = a; }
      void showstudent()
     {
        Cout << "\n Name: " << name;
        Cout << "\n Age: " << age; }
     };

int main()
{
Pair <string, int > P1;
Pair <string, string > P2;
Pair <string, float > P3;
*Pair < int , student > P4;
```

```
P1 = make_Pair("Rahul", 16);
P2 = make_Pair("India", "New Delhi");
P3 = make_Pair("Drilling c++", 545.51);

student s1;
s1.setstudent("Aishvarya", 19);
P4 = make_Pair(1, s1);

cout<<"\n Pair 1";
cout<<"\n"<<P1.first<<" "<<P1.second;
                Print first value   Print second value
cout<<"\n Pair 2";
cout<<"\n"<<P2.first<<" "<<P2.second;

cout<<"\n Pair 3";
cout<<"\n"<<P3.first<<" "<<P3.second;

cout<<"\n Pair 4";
cout<<"\n"<<P4.first<<" ";
  student s2 = P4.second; → it is returning student object
  s2.showstudent();       ↓ you can also use operator overloading
}
```

Compare two pairs

- == → if two pairs are equal or not
- !=
- <
- >
- <=
- >=

Tuple in STL

Pair → we can pair two heterogenous objects
Tuple → we can pair multiple objects

ex:-
```
tuple <string, int, int> t1;

t1 = make_tuple ("india", 16, 10);
```

```
#include <iostream>
#include <tuple>
using namespace std;
int main()
{
    tuple <string, int, int> t1;

    t1 = make_tuple ("Rahul", 15, 36);

    cout << get<0>(t1) << " ";
    cout << get<1>(t1) << " ";
    cout << get<2>(t1) << " ";
```

↓

Rahul 15 3

Compare two tupks

- ==
- !=
- <
- >
- <=
- >=

0755 - 405105

Vector class

↓
it supports a Dynamic Array & array class is static

```
#include <iostream>
#include <vector>
using namespace std;

int main()
{
    vector <int> v1;  → Creates vector of zero space

    vector <int> v1 = {2, 3, 5}; → Creates vector of 3 space & initialise them with 2, 3, 5

    vector <char> v2 (4); → Create vector of 4 space
```

*
```
    vector <char> cv(1, 'a');
    vector <int> v3 (5, 10);
    vector <string> v4 (3, "hello")
```

↓ 0	1	2
hello	hello	hello

Relational operators

- ==
- !=
- >
- <
- >=
- <=

[ ]
↓

cout << v4[0]; → hello
cout << v4[1]; → hello

Push_back() Pop_back() Capacity()

vector<int> v1 = {2,3,5};

v1 | 2 | 3 | 5 |

v1.Push_back(10) → double the size after filling

v1 | 2 | 5 | 5 | 10 | | |

size()

```
For (int i=0; i<=9; i++)
    v1.Push_back(10*(i+1));

For (int i=0; i<v1.size(); i++)
    cout << v1[i] << endl;
```

→
10
20
30
40
50
60
70
80
90
100

v1.clear() → it clears all the elements of vector

at() → Prints value at given index
index

Front() → Prints First element of vector

back() → Prints last element of vector

iterator

Previous code
+

```
For ( int i=0; i< v1.size(); i++)
    cout << v1[i] << endl;
```

```
vector < int > :: iterator it;
    it = v.begin()
```

For inserting at particular place

```
v1.insert ( it+2, 35);

For ( int i=0; i< v1.size() ; i++)
    cout << v1[i];
```

```
v1.insert( it, 35);
```

35
10
20
30
40
50
60
70
80
90
100

0 10
1 20
2 35
3 30
4 40
5 50
60
70
80
90
100

List class

↓

List can be accessed Front to back OR back to front.

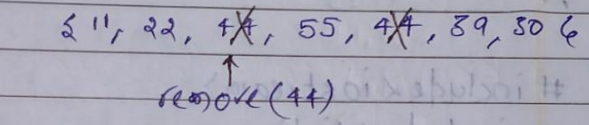
** vector support random access but a list can be accessed sequentially only.

[X] → doesn't work here

```
#include<iostream>
#include<list>
  using namespace std;

  int main()
{
    list<int> l1 {11, 22, 33};

    list<string> l2 {"India", "Kathmandu", "Islamabad", "Dhaka"};

    list<int>::iterator P = l1.begin();

      while(P != l1.end())
    {   cout << *P << " ";      → 11, 22, 33
        P++;
    }
    cout << "Total values in the list are" << l1.size();
                                              ↓
                                              4
```

Useful function of list class

- sort()
- size()
- Push_back()
- Push_front()
- POP_back()
- POP_front()
- reverse()
- remove(x)
- clear



map class

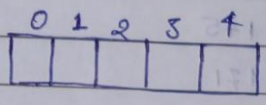
↓

it is used to replicate associative arrays

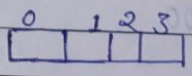
Array are of two types

① Numeric → Arrays whose index is ~~zero~~ integer & starts from 0.

int a[5];

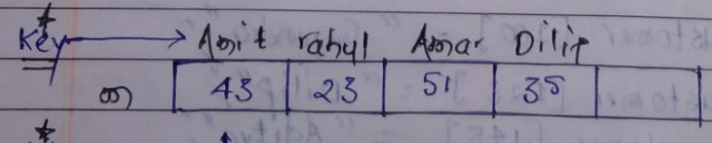


float b[4];



string s[3];

② Associative :→ we can represent index by name, string, integer



∞ ["Rahul"] → 23

Key ↓	value ↓
Amit	43
Rahul	23
Amar	51
Dilip	35

* each key should be unique.

* maps always arrange its keys in sorted order
  ↳ in case of string → dictionary order

Ex:-

Customer Number	Customer Name
100	Gajendra
123	Dilip
145	Aditya
171	shahid
200	Rajesh

→

```
# include <iostream>
# include <map>
using namespace std;

int main()
{
    map<int, string> customer;

1st way of storing ←
    customer [100] = "Gajendra";
    customer [128] = "Dilip";
    customer [145] = "Aditya";
    customer [171] = "shahid";
    customer [200] = "Rajesh";
```

2nd way of storing

```
map<int, string> c {{100, "Gajendra"}, {123, "pitip"}
    {145, "Aditya"}, {171, "shahid"}, {200, "Rajesh"}};
```

Useful functions

- at()
- []
- size()
- empty() → To check map is empty or not
- insert()
- clear()

iterator

```
map<int, string> :: iterator P = customer.begin();
while (P != customer.end())
{
    cout << P->first << " " << P->second << endl;
    P++;
}
```

at()

```
cout<<customer.at(145);
```
→ Aditya

insert()

```
customer.insert(pair<int,string>(205, "saurabh"));
```

→ unorder_map → it's internal working based on Hash table

↓

Avg case → O(1)

→ map → based on binary search tree

↓

order of logn

* Try to use unorder_map